Rockchip RK3399 Linux SDK Quick Start

ID: RK-FB-YF-944

Release Version: V1.4.0

Release Date: 2023-12-20

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

Trademark Statement

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document presents Rockchip RK3399 Linux SDK release notes, aiming to help engineers get started with RK3399 Linux SDK development and debugging faster.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Chipset and System Support

Chipset	Buildroot	Debian	Yocto
RK3399	Y	Y	Y

Revision History

Date	Version	Author	Revision History
2022-06-20	V1.0.0	Caesar Wang	Initial version
2022-09-20	V1.0.1	Caesar Wang	Update it for linux5.10
2022-11-20	V1.0.2	Caesar Wang	Update Linux Upgrade Instruction
2023-04-20	V1.1.0	Caesar Wang	Adapt to the new version of SDK
2023-05-20	V1.1.1	Caesar Wang	Update SDK to V1.1.1
2023-06-20	V1.2.0	Caesar Wang	Update SDK to V1.2.0
2023-09-20	V1.3.0	Caesar Wang	Update SDK to V1.3.0
2023-12-20	V1.4.0	Caesar Wang	Update SDK to V1.4.0

Contents

1. SDK Precompiled Image

By using the RK3399 Linux SDK precompiled image, developers can bypass the process of compiling the entire operating system from source code. Instead, they can directly flash the precompiled image into the RK3399 development board, enabling rapid start of development and related evaluations and comparisons. This can reduce development time and cost wastage caused by compilation issues.

The firmware can be downloaded from the public address Click here for SDK firmware download.

Firmware path: General Linux SDK Firmware -> Linux5.10 -> RK3399

If you need to modify SDK code or for a quick start, please refer to the following chapters.

2. Set up an Development Environment

2.1 Preparing the development environment

It is recommended to use Ubuntu 22.04 for compilation. Other Linux versions may need to adjust the software package accordingly. In addition to the system requirements, there are other hardware and software requirements. Hardware requirements: 64-bit system, hard disk space should be greater than 40G. If you do multiple builds, you will need more hard drive space

2.2 Install libraries and toolsets

When using the command line for device development, you can install the libraries and tools required for compiling the SDK by the following steps.

Use the following `apt-get` command to install the libraries and tools required for the following operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

> **Description:**
> The installation command is applicable to Ubuntu22.04. For other versions, please use the corresponding installation command according to the name of the installation package. If you encounter an error when compiling, you can install the corresponding software package according to the error message. in:
>
> - Python 3.6 or later versions is required to be installed, and python 3.6 is used as an example here.
> - make requires make 4.0 and above to be installed, take make 4.2 as an example here.
> - lz4 1.7.3 or later versions is required to be installed.
> - Compiling yocto requires a VPN network, and git does not have the CVE-2022-39253 security detection patch.

2.2.1 Check and upgrade the `python` version of the host

The method of checking and upgrading the `python` version of the host is as follows:

- Check host `python` version

```
$ python3 --version
Python 3.10.6
```

If you do not meet the requirements of python>=3.6 version, you can upgrade it in the following way:

- Upgrade `python 3.6.15` new version

```
PYTHON3_VER=3.6.15
echo "wget
••https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Check and Upgrade the `make` Version on the Host

The method to check and upgrade the `make` version on the host is as follows:

- Check the `make` version on the host

```
$ make -v/
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrade to the new version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Check and Upgrade the `lz4` Version on the Host

The method to check and upgrade the `lz4` version on the host is as follows:

- Check the `lz4` version on the host

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
refusing to read from a console
```

- Upgrade to the new version of `lz4`

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

2.2.4 Check and Upgrade the `git` Version on the Host

- Check the `git` version on the host

```
$ /usr/bin/git -v
git version 2.38.0
```

- Upgrade to the new version of `git`

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

3. Docker Environment Setup

To help developers quickly complete the complex development environment preparation mentioned above, we also provide a cross-compiler Docker image. This enables customers to rapidly verify and then shorten the build time of the compilation environment.

Before using the Docker environment, you can refer to the following document for instructions: `<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf`.

The following systems have been verified:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
ubuntu 22.04	24.0.5	pass	pass
ubuntu 21.10	20.10.12	pass	pass
ubuntu 21.04	20.10.7	pass	pass
ubuntu 18.04	20.10.7	pass	pass
fedora35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the website Docker Image.

4. Software Development Guide

4.1 Development Guide

Aiming to help engineers get started with SDK development and debugging faster, We have released “Rockchip_Developer_Guide_Linux_Software_EN.pdf” with the SDK, please refer to the documents under the project's `docs/en/RK3399` directory.

4.2 Chip Datasheet

Aiming to help engineers get started with RK3399 development and debugging faster. We have released "Rockchip_RK3399_Datasheet_V2.1_20200323.pdf" , please refer to the documents under the project's `docs/en/RK3399/Datasheet` directory.

4.3 Debian Development Guide

Aiming to help engineers get started with RK3399 Debian development and debugging faster, “Rockchip_Developer_Guide_Debian_EN.pdf” is released with the SDK, please refer to the documents under the project's `docs/en/Linux/System` directory, which will be continuously improved and updated.

4.4 Third Party System Adaptation

Aiming to help engineers get started with Third Party System Adaptation faster, “Rockchip_Developer_Guide_Third_Party_System_Adaptation_EN.pdf” is released with the SDK, please refer to the documents under the project's `docs/en/Linux/System` directory, which will be continuously improved and updated.

4.5 Software Update History

Software release version upgrade history can be checked through project xml file by the following command:

```
.repo/manifests$ realpath rk3399_linux5.10_release.xml
# e.g.:the printed version is v1.4.0 and the update time is 20231220
# <SDK>/.repo/manifests/rk3399_linux/rk3399_linux5.10_release_v1.4.0_20231220.xml
```

Software release version updated information can be checked through the project text file by the following command:

```
<SDK>/.repo/manifests/rk3399_linux/RK3399_Linux5.10_SDK_Note.md
or
<SDK>/docs/en/RK3399/RK3399_Linux5.10_SDK_Note.md
```

5. Hardware Development Guide

Hardware related development can refer to the user guide, which can be found in the engineering directory:

RK3399 Hardware Design Guidelines:

```
<SDK>/docs/en/RK3399/Hardware/Rockchip_RK3399_Hardware_Design_Guide_V1.2_EN.pdf
```

RK3399 IND Industry Board Hardware Development Guide:

```
<SDK>/docs/en/RK3399/Hardware/Rockchip_RK3399_User_Manual_IND_EVB_V1.1_EN.pdf
```

6. The Precaution of IO Power Design

The IO level of the controller power domain must be consistent with the IO level of the connected peripheral chip, and the voltage configuration of software must be consistent with the voltage of hardware to avoid GPIO damage.

 Note:

About matching of GPIO power domain and IO level:

PMUIO0_VDD, PMUIO1_VDD, VCCIO1_VDD, VCCIO2_VDD, VCCIO3_VDD, VCCIO4_VDD, VCCIO5_VDD, VCCIO6_VDD, VCCIO7_VDD, voltage of these GPIO power domain must be consistent with the IO level voltage of the connected peripheral to avoid GPIO damage.

Also need to note that the voltage configuration of software should be consistent with the voltage of hardware: For example, if hardware IO level is connected to 1.8V, the voltage configuration of software should be configured to 1.8V accordingly; if hardware IO level should be connected to 3.3V, and the voltage configuration of software should also be configured to 3.3V to avoid GPIO damage.

Please refer to the following documents for details:

```
<SDK>/docs/en/RK3399/Rockchip_RK3399_Introduction_IO_Power_Domains_Configuration.pdf
<SDK>/docs/en/Common/IO-DOMAIN/Rockchip_Developer_Guide_Linux_IO_DOMAIN_EN.pdf
```

7. SDK Configuration Framework Introduction

7.1 SDK Project Directory Introduction

There are buildroot, debian, recovery, app, kernel, u-boot, device, docs, external and other directories in the project directory. Repositories are managed using manifests, and the repo tool is used to manage each directory or its subdirectory corresponding to a git.

- buildroot: root file system based on Buildroot.
- debian: root file system based on Debian.
- device/rockchip: store board-level configuration for each chip and some scripts and prepared files for building and packaging firmware.
- docs: stores development guides, platform support lists, tool usage, Linux development guides, and so on.
- external: stores some third-party libraries, including audio, video, network, recovery and so on.
- kernel: stores kernel development code.
- output: stores the firmware information, compilation information, XML, host environment, etc. generated each time.
- prebuilts: stores cross-building toolchain.
- rkbin: stores Rockchip Binary and tools.
- rockdev: stores building output firmware.
- tools: stores some commonly used tools under Linux and Windows system.
- u-boot: store U-Boot code developed based on v2017.09 version.
- yocto: stores the root file system developed based on Yocto 4.0.

8. SDK Building Introduction

The SDK can be accessed through `make` or `./build.sh` , add target parameters to configure and compile related functions.

8.1 SDK Compilation Commands

`make help` , e.g:

The SDK can configure and compile relevant features by using `make` or `./build.sh` with target parameters. Please refer to the `device/rockchip/common/README.md` compilation instructions for details.

8.2 SDK Compilation Command View

`make help`, for example:

```
$ make help
  menuconfig             - interactive curses-based configurator
  oldconfig              - resolve any unresolved symbols in .config
  syncconfig             - Same as oldconfig, but quietly, additionally update
deps
  olddefconfig           - Same as syncconfig but sets new symbols to their
default value
  savedefconfig          - Save current config to RK_DEFCONFIG (minimal config)
  ...
```

The actual operation of make is `./build.sh`

You can also run `./build.sh <target>` to compile the relevant functions, which can be done through `./build.sh help` View the specific compilation commands.

```
$ ./build.sh -h
################ Rockchip Linux SDK ###############

Manifest: rk3399_linux_release_v1.4.0_20231220.xml

Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]           choose chip
defconfig[:<config>]               choose defconfig
config                             modify SDK defconfig
...
updateimg                          build update image
otapackage                         build A/B OTA update image
all                                build images
release                            release images and build info
save                               alias of release
all-release                        build and release images
allsave                            alias of all-release
```

```
shell                                setup a shell for developing
cleanall                             cleanup
clean[:module[:module]]...           cleanup modules
post-rootfs <rootfs dir>             trigger post-rootfs hook scripts
help                                 usage

Default option is 'all'.
```

8.3 SDK board-level configuration

Enter the project `<SDK>/device/rockchip/rk3399` directory:

Board level configuration	Note
rockchip_rk3399_evb_ind_lpddr4_defconfig	Suitable for RK3399 industry development board
rockchip_rk3399_firefly_defconfig	Suitable for RK3399 firefly development boards
rockchip_rk3399_sapphire_excavator_lp4_defconfig	Suitable for RK3399 sapphire excavator with LPDDR4 development board
rockchip_rk3399_sapphire_excavator_defconfig	Suitable for RK3399 sapphire excavator development board

The first way:

Add board configuration file behind `/build.sh` , for example:

Select the board configuration of **RK3399 industry development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_evb_ind_lpddr4_defconfig
```

Select the board configuration of the **RK3399 firefly development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_firefly_defconfig
```

Select the board-level configuration of the **RK3399 sapphire excavator with LPDDR4 development board**:

```
./build.sh
device/rockchip/rk3399/rockchip_rk3399_sapphire_excavator_lp4_defconfig
```

Select the board-level configuration of the **RK3399 sapphire excavator development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_sapphire_excavator_defconfig
```

The second way:

```
rk3399$ ./build.sh lunch
processing option: lunch

You're building on Linux
Lunch menu...pick a combo:
```

```
0. default BoardConfig.mk
1. BoardConfig-rk3399-evb-ind-lpddr4.mk
2. BoardConfig-rk3399-firefly.mk
3. BoardConfig-rk3399-sapphire-excavator-lp4.mk
4. BoardConfig-rk3399-sapphire-excavator.mk
5. BoardConfig.mk
Which would you like? [0]:
...
```

8.4 SDK Customization Configuration

The SDK can be configured through `make menuconfig`, with the main components currently available for configuration being:

```
(rk3399) SoC
    Rootfs  --->
    Loader (u-boot)  --->
    RTOS  --->
    Kernel  --->
    Boot  --->
    Recovery (based on buildroot)  --->
    PCBA test (based on buildroot)  --->
    Security  --->
    Extra partitions  --->
    Firmware  --->
    Update (Rockchip update image)  --->
    Others configurations  --->
```

- Rootfs: This represents the "Root File System". Here, you can choose different root file systems like Buildroot, Yocto, Debian, etc.
- Loader (u-boot): This is the boot loader configuration, usually u-boot, which initializes hardware and loads the main operating system.
- RTOS: Real-Time Operating System (RTOS) configuration options, suitable for applications requiring real-time performance.
- Kernel: Configure kernel options here, customizing a Linux kernel suitable for your hardware and application needs.
- Boot: Configure Boot partition support formats here.
- Recovery (based on buildroot): This is the configuration for the recovery environment based on buildroot, used for system recovery and upgrade.
- PCBA test (based on buildroot): This is a configuration for a PCBA (Printed Circuit Board Assembly) testing environment based on buildroot.
- Security: Indicates that certain security features will depend on the buildroot configuration chosen for Rootfs, mainly the Secureboot feature activation.
- Extra partitions: Used to configure additional partitions.
- Firmware: Configure firmware-related options here.
- Update (Rockchip update image): Used to configure options for Rockchip's complete firmware.
- Others configurations: Other additional configuration options.

The `make menuconfig` interface provides a text-based user interface to select and configure various options. Once configuration is complete, use the command `make savedefconfig` to save these settings, so that custom builds will be done based on these settings.

With the above configs, you can choose different Rootfs/Loader/Kernel configurations for various custom builds, allowing flexible selection and configuration of system components to meet specific needs.

8.5 Automatic Build

Enter root directory of project directory and execute the following commands to automatically complete all build:

```
./build.sh all # Only build module code(u-Boot, kernel, Rootfs, Recovery)
               # Need to execute ./mkfirmware.sh again for firmware package

./build.sh     # Base on ./build.sh all
               # 1. Add firmware package ./mkfirmware.sh
               # 2. update.img package
               # 3. Save the patches of each module to the out directory
               # Note: ./build.sh  and  ./build.sh allsave command are the same
```

It is Buildroot by default, you can specify rootfs by setting the environment variable RK_ROOTFS_SYSTEM. There are two types of system for RK_ROOTFS_SYSTEM: buildroot and debian.

If you need debain, you can generate it with the following command:

```
export RK_ROOTFS_SYSTEM=debian
./build.sh
or
RK_ROOTFS_SYSTEM=debian ./build.sh
```

> Note:
> Every time the SDK is updated, it is recommended to clean up the previous compiled products and run them directly `./build.sh cleanall`

8.6 Build and Package Each Module

8.6.1 U-boot Build

```
./build.sh uboot
```

8.6.2 Kernel Build

- Method 1

```
./build.sh kernel
```

- Method 2

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-
x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
or
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
```

- Method 3

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
or
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
```

8.6.3 Recovery Build

```
./build.sh recovery
```

Note: Recovery is a unnecessary function, some board configuration will not be set

8.6.4 Buildroot Build

Enter project root directory and run the following commands to automatically complete compiling and packaging of Rootfs.

```
./build.sh rootfs
```

After compilations, rootfs.ext4 is generated in Buildroot directory “ output/rockchip_rk3399/images”.

8.6.5 Debian Build

```
 ./build.sh debian
```

After compilation, generate linaro-rootfs. img in the Debian directory.

> Description：re-install the depend packages
> sudo apt-get install binfmt-support qemu-user-static live-build
> sudo dpkg -i ubuntu-build-service/packages/*
> sudo apt-get install -f

For specific details, please refer to Debian development documentation reference:

`<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Debian_EN.pdf`

8.6.6 Yocto Build

Enter project root directory and execute the following commands to automatically complete compiling and packaging Rootfs.

EVB boards：

```
./build.sh yocto
```

After compiling, rootfs.img is generated in yocto directory “/build/lastest”.
The default login username is root.

Please refer to Rockchip Wiki for more detailed information of Yocto.

FAQ：

- If you encounter the following problem during above compiling:

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).
Python can't change the filesystem locale after loading so we need a UTF-8
when Python starts or things won't work.
```

Solution:

```
locale-gen en_US.UTF-8
export LANG=en_US.UTF-8 LANGUAGE=en_US.en LC_ALL=en_US.UTF-8
```

Or refer tosetup-locale-python3.

8.6.7 Cross-Compilation

8.6.7.1 SDK Directory Built-in Cross-Compilation

The SDK prebuilts directory built-in cross-compilation are as follows:

Contents	Description
prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	gcc arm 10.3.1 64-bit toolchain
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf	gcc arm 10.3.1 32-bit toolchain

You can download the toolchain from the following address:

Click here

8.6.7.2 Buildroot Built-in Cross-compilation

The configuration of different chips and target functions can be set through `source buildroot/envsetup.sh`

```
$ source buildroot/envsetup.sh
Top of tree: rk3399

Pick a board:
24. rockchip_rk3399
25. rockchip_rk3399_base
26. rockchip_rk3399_recovery


Which would you like? [1]:
```

Default selection 24 `rockchip_rk3399`, then enter the Buildroot directory for RK3399 and start compiling the relevant modules.

For example, to compile the rockchip-test module, commonly used compilation commands are as follows:

Enter the buildroot directory

```
SDK#cd buildroot
```

- Delete and recompile rockchip-test

```
buildroot#make rockchip-test-recreate
```

- Recompile rockchip-test

```
buildroot#make rockchip-test-rebuild
```

- Delete rockchip-test

```
buildroot#make rockchip-test-dirclean
or
buildroot#rm -rf output/rockchip_rk3399/build/rockchip-test-master/
```

If you need to compile a single module or a third-party application, you need to configure the cross-compilation environment. For example, RK3399 its cross-compilation tool is located in the `buildroot/output/rockchip_rk3399/host/usr` directory, you need to set the bin/ directory of the tool and the `aarch64-buildroot-linux-gnu/bin/` directory as the environment variable, execute the script that automatically configures environment variables in the top-level directory::

```
source buildroot/envsetup.sh rockchip_rk3399
```

Enter the command to view:

```
cd buildroot/output/rockchip_rk3399/host/usr/bin
./aarch64-linux-gcc --version
```

The following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot) 12.3.0
```

> Save to rootfs configuration file
> buildroot$ make update-defconfig

8.6.8 Firmware Package

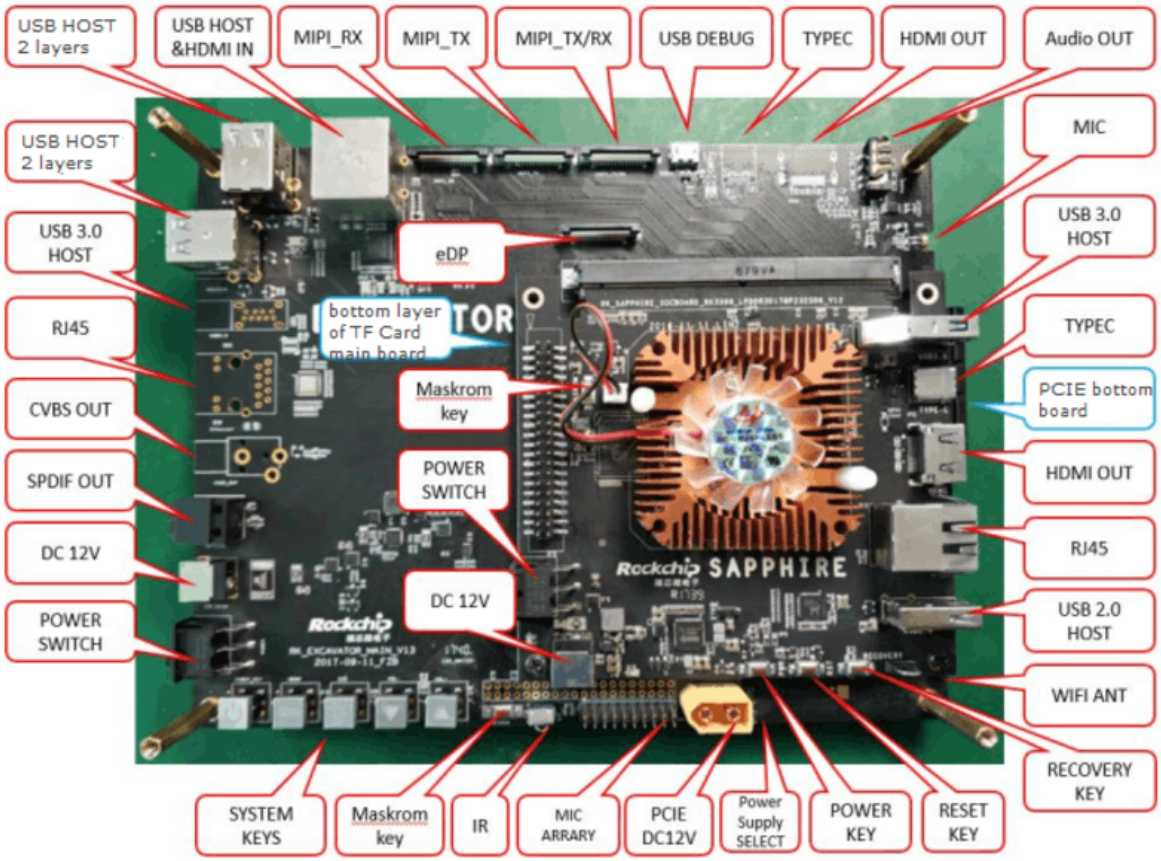
After compiling various parts of Kernel/U-Boot/Recovery/Rootfs above, enter root directory of project directory and run the following command to automatically complete all firmware packaged into `output/firmware` directory:

Firmware generation:

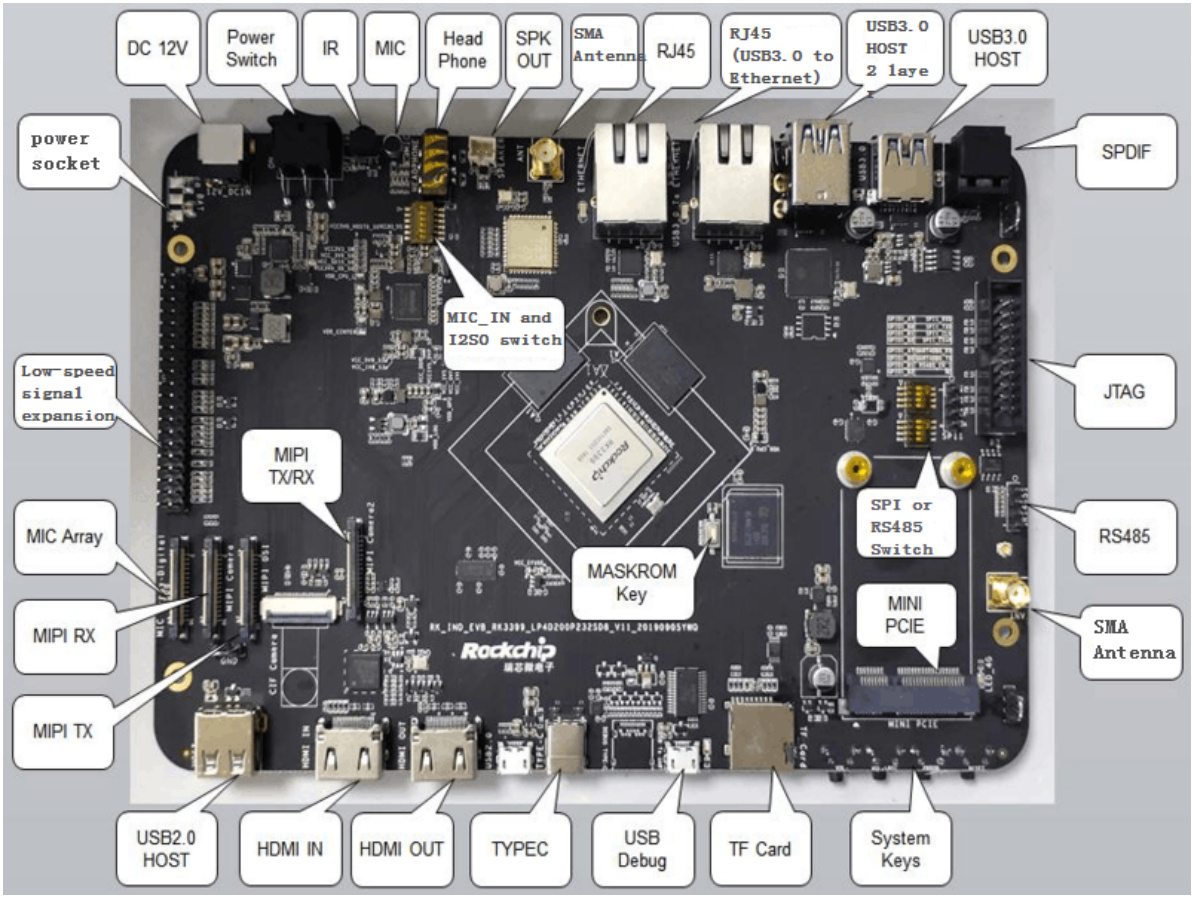
```
./build.sh firmware
```

9. Upgrade Introduciton

Interfaces layout of RK3399 excavator are showed as follows:



Interfaces layout of RK3399 IND industry board are showed as follows:

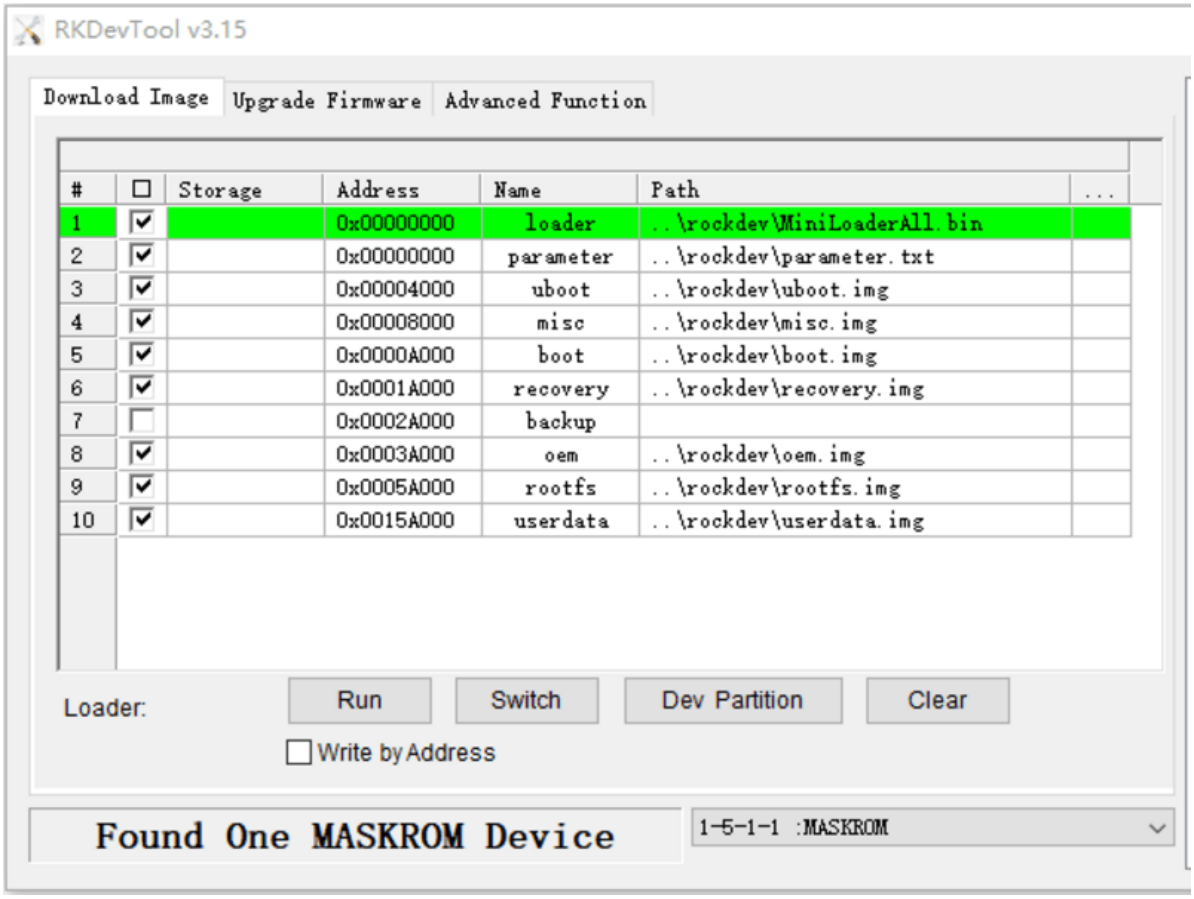


9.1 Windows Upgrade Introduction

SDK provides windows upgrade tool (this tool should be V3.15or later version) which is located in project root directory:

```
tools/
├── windows/RKDevTool
```

As shown below, after compiling the corresponding firmware, device should enter MASKROM or BootROM mode for update. After connecting USB cable, long press the button “MASKROM” and press reset button “RST” at the same time and then release, device will enter MASKROM Mode. Then you should load the paths of the corresponding images and click “Run” to start upgrade. You can also press the “recovery” button and press reset button “RST”then release to enter loader mode to upgrade. Partition offset and flashing files of MASKROM Mode are shown as follows (Note: Window PC needs to run the tool as an administrator):



Note：Before upgrade, please install the latest USB driver, which is in the below directory:

```
<SDK>/tools/windows/DriverAssitant_v5.12.zip
```

9.2 Linux Upgrade Instruction

The Linux upgrade tool (Linux_Upgrade_Tool should be v2.17 or later versions) is located in “tools/linux” directory. Please make sure your board is connected to MASKROM/loader rockusb, if the compiled firmware is in rockdev directory, upgrade commands are as below:

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -trust rockdev/trust.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rocdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

Or upgrade the whole update.img in the firmware

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or in root directory, run the following command on the machine to upgrade in MASKROM state:

```
./rkflash.sh
```

9.3 System Partition Introduction

Default partition introduction (below is RK3399 IND reference partition):

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4096K	uboot
2	24576	32767	4096K	trust
3	32768	40959	4096K	misc
4	40960	106495	32M	boot
5	106496	303104	32M	recovery
6	172032	237567	32M	bakcup
7	237568	368639	64M	oem
8	368640	12951551	6144M	rootfs
9	12951552	30535646	8585M	userdata

- uboot partition: for uboot.img built from uboot.
- trust partition: for trust.img built from uboot.
- misc partition: for misc.img built from recovery.
- boot partition: for boot.img built from kernel.
- recovery partition: for recovery.img built from recovery.
- backup partition: reserved, temporarily useless. Will be used for backup of recovery as in Android in future.
- oem partition: used by manufactor to store their APP or data, mounted in /oem directory
- rootfs partition: store rootfs.img built from buildroot or debian.
- userdata partition: store files temporarily generated by APP or for users, mounted in /userdata directory.